Project Overview

1. What Is This Project

A chatbot that lets non-technical users query their own business data using plain English. No SQL knowledge needed — the user types a question and gets a direct answer in seconds.

The system handles two types of questions:

- Questions about live data (e.g. "How many failed transactions this month?") — answered by generating and running a SQL query against the database.
- Questions about trends and insights (e.g. "What are the payment trends?") — answered by searching pre-generated insight documents.

Anything outside the scope of business data is blocked. The system runs on 250,000 real UPI transaction records and is deployed across three free-tier cloud services.

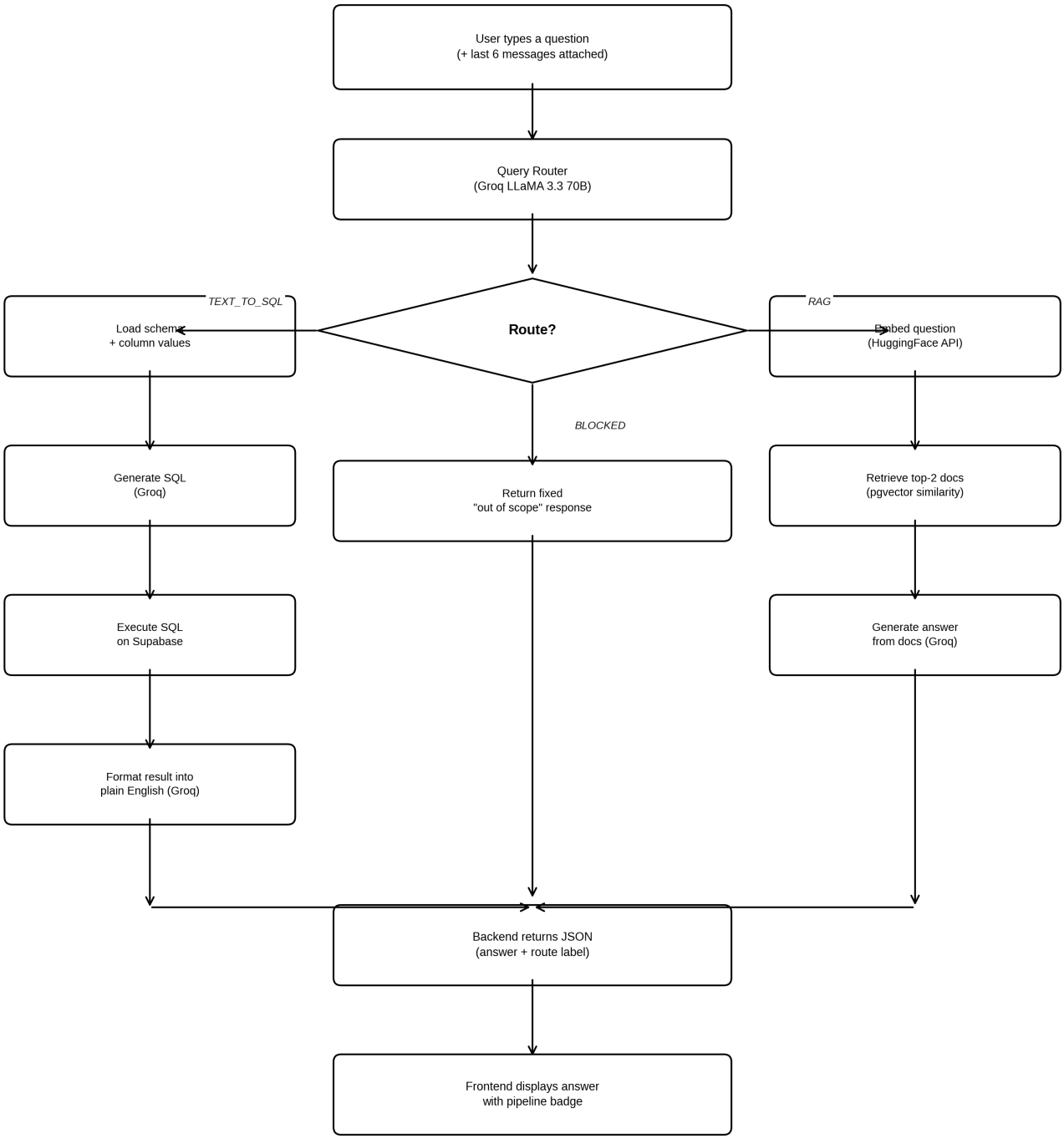
2. System Architecture

Three physical layers, one logical layer:

Layer	What It Does
Frontend (Vercel)	React + TypeScript chat interface. Manages session state with Zustand (persisted in localStorage). Sends every message with user ID and last 6 conversation turns to the backend.
Backend (Render)	FastAPI Python app. Two endpoints: /chat for questions, /upload for file ingestion. All AI logic lives here. No local ML models — everything calls external APIs.
Database (Supabase)	Hosted PostgreSQL with pgvector extension. Stores both relational transaction data and vector embeddings for RAG. One service handles both — no separate vector DB needed.
AI Routing (logical)	Lives inside the backend. Classifies each question into TEXT_TO_SQL, RAG, or BLOCKED before any pipeline runs.

3. Execution Flow

The diagram below shows what happens from the moment a user sends a message to the moment an answer is displayed.



Key things to note about the flow:

- The router always runs first — it sees the full conversation history so follow-up questions like "and on weekends?" are correctly understood in context.

- BLOCKED returns a fixed response immediately — no LLM or database call happens.
- The schema fed to the SQL generator includes the actual column values from the data (e.g. it knows the status column says SUCCESS, not PASS). This prevents the LLM from generating queries that return zero rows due to wrong assumptions.
- RAG documents are pre-generated at server startup by running aggregation queries over the user's data and asking Groq to write insight summaries. They are not generated per request.

4. Key Components

Backend

- main.py — Entry point. Defines /chat and /upload endpoints. On first startup: creates tables, generates RAG documents, warms schema cache.
- query_router.py — Sends question + history to Groq, gets back TEXT_TO_SQL / RAG / BLOCKED.
- schema_loader.py — Fetches table names, columns, and up to 20 distinct values per column. Cached in memory after first call. Invalidated on new CSV upload.
- sql_generator.py — Sends schema + question to Groq. Instruction: return raw SQL only, always filter by user_id, return CANNOT_ANSWER if data does not exist.
- sql_executor.py — Runs the SQL on Supabase via SQLAlchemy. Returns raw list of dicts.
- generate_insights.py — Runs ~10 aggregation queries per user on startup. Sends results to Groq to write insight paragraphs. These become the RAG document set.
- embedder.py — Sends document text to HuggingFace API, gets 384-dim vector, upserts into rag_documents table.
- retriever.py — Embeds the user's question, runs cosine similarity search in pgvector, returns top 2 matching docs.
- groq_client.py — Thin wrapper. All LLM calls route through here. Handles message history formatting.
- file_processor.py — Handles CSV (creates new Supabase table), PDF (PyPDF2 text extraction), and TXT uploads. Text files are chunked and embedded as RAG documents.

Frontend

- chatStore.ts — Zustand store. Holds all session state, auth state, theme. Sessions persist to localStorage. Auth re-fetched from Supabase on each page load.
- api.ts — Two functions: sendChatMessage (POST to /chat) and uploadFile (multipart POST to /upload). Base URL from environment variable.

5. External Services & Key Dependencies

Service / Library	Role
Groq (LLaMA 3.3 70B)	Query routing, SQL generation, insight writing, result formatting. Chosen for speed and free-tier 128k context window.
HuggingFace Inference API	Text embeddings (all-MiniLM-L6-v2, 384 dims). Called as REST API to avoid installing PyTorch locally (900MB+ — exceeds Render's 512MB RAM limit).
Supabase (PostgreSQL)	Primary database + vector store. Persists across Render restarts. pgvector extension replaces a separate vector DB like ChromaDB.

FastAPI + SQLAlchemy	Backend framework and ORM. psycopg2-binary used for zero native-dependency PostgreSQL connections.
React 19 + TypeScript	Frontend framework. TypeScript enforces API response shape contracts.
Zustand	Frontend state management. Chosen for minimal API and built-in localStorage persistence.
Tailwind + shadcn/ui	Styling and component library.
Vercel / Render	Frontend and backend deployment. Both on free tier.

6. Why It Works This Way

- Two pipelines instead of one — SQL and RAG answer structurally different question types. Forcing one through the other would either slow down data queries (embedding every question) or make insight questions brittle (schema-dependent).
- No local ML models — Render free tier allows 512MB RAM. Sentence-transformer + PyTorch alone is ~900MB. HuggingFace API offloads the compute entirely.
- Supabase instead of SQLite + ChromaDB — Render has an ephemeral filesystem. Any file written to disk is wiped on restart. Supabase is hosted and persistent, and pgvector removes the need for a second vector service.
- Schema caching — Fetching schema per request adds a full Supabase round-trip before any SQL is generated. Caching eliminates this after the first request.
- Conversation history in routing — Without it, "what about on weekends?" would be misclassified. Passing the last 6 turns to the router makes follow-up questions work correctly.